RV UNIVERSITY, BENGALURU-59

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

A Mini Project Report On

Hinglish Smart Writing Assistant

Transliteration, Phrasing Improvement and Translation of Roman-Script Hinglish

Submitted in partial fulfillment for the award of the degree of
B.Tech. (Honors)
in
School of Computer Science and Engineering

Submitted By

Rakshitha K	1RVU23CSE369
R Keerthana Prasad	1RVU23CSE363
N Sukhitha Bhushan	1RVU23CSE293
SathyaShree KV	1RVU23CSE412

Under the Guidance of
Prof. Harikumar Vasudevan Pillai Santhibhavan
Professor of Practice
School of CSE
RV University, Bengaluru-560059

2026

LIST OF TABLES

Table No	Table Name
2.1	Summary of resources reviewed
3.1	Hardware and software configuration used
5.1	Test set categories
5.2	Libraries used in the project
5.3	Tools used in the project
5.4	Dependency problems and solutions
6.1	Baseline results by category (normalisation off)
6.2	Results by category with normalisation
6.3	Overall results, baseline versus normalised
6.4	Strict CER by category, before and after
6.5	Representative examples
6.6	Failure categories
A2.1	The 40 test cases

LIST OF FIGURES

Figure No	Figure Name
Fig 4.1	System architecture (pipeline)
Fig 6.1	Strict CER by category, baseline versus normalised
Fig A1.1	Start screen of the application
Fig A1.2	Example with phrasing off
Fig A1.3	Example with phrasing on (fallback)

ABSTRACT

Many Indian users type Hindi in Roman script, a style commonly called Hinglish. This text is hard to process because spellings are not standardised, English words are mixed freely into Hindi sentences, and the Roman script carries no information about vowel length, gender or diacritics. This report describes the design, implementation and evaluation of the Hinglish Smart Writing Assistant, an end-to-end application that converts Roman-script Hinglish into Devanagari Hindi, optionally improves the phrasing, and optionally translates the result into English. The system chains AI4Bharat IndicXlit for transliteration and the distilled 200M IndicTrans2 model for translation, runs on a CPU-only laptop, and is exposed through a Gradio web interface. An optional phrasing step calls a hosted language model through a swappable backend. A test set of 40 hand-checked sentences in seven categories was used to measure character error rate (CER), word accuracy, chrF and BLEU. The baseline pipeline reached a strict CER of 7.00% and a word accuracy of 81.85%. A small rule-based normalisation layer for informal spellings and numerals reduced the strict CER to 4.88% and raised word accuracy to 86.29%, with the largest gains in the spelling and numbers categories. Remaining errors involve proper names, English loanwords and ambiguous words, and are analysed in detail.

Keywords: Hinglish, transliteration, machine translation, IndicXlit, IndicTrans2, character error rate, Gradio.

CHAPTER 1: INTRODUCTION

A large share of everyday digital communication in India is written in Roman script even though the underlying language is Hindi. A message such as “mujhe kal university jaana hai” is easy for a Hindi speaker to read but difficult for software that expects either standard English or standard Devanagari text. In this project, pretrained AI4Bharat models are combined with a small normalisation layer and an optional language-model step to convert such text into Devanagari, improve its phrasing and translate it into English.

1.1 STATE OF THE ART

Recent work on Indian-language text processing relies on transformer-based neural models trained on large multilingual corpora. AI4Bharat has released open models for transliteration (IndicXlit) and for translation between Indian languages and English (IndicTrans2), together with datasets such as Aksharantar for transliteration research. Large language models are increasingly used for rewriting and improving text. Remaining challenges include informal spelling, code-mixing with English, proper names, and the lack of gender and diacritic information in Roman script.

1.2 MOTIVATION

Roman-script Hinglish is how many Indians type on phones and social media. Converting it into proper Devanagari, cleaning up its phrasing and translating it makes writing tools, search and accessibility applications more useful for Hindi speakers. Building such a tool from pretrained models also shows how far open models can go on a standard laptop without specialised hardware.

1.3 PROBLEM STATEMENT

Roman-script Hinglish is ambiguous. The same word has many spellings (nahi, nahin, nhi), English words are mixed into Hindi sentences, and the script does not mark gender or vowel length, so a word such as “naya” cannot be resolved to “नया” or “नई” without grammatical context. Errors made when converting to Devanagari also carry over into any later translation step. The task is to build a usable, end-to-end application that converts Hinglish into native script, improves the phrasing, and optionally translates into English, with measurable evaluation.

1.4 OBJECTIVES

The main objectives of this project are:

- To build a pipeline of pretrained AI4Bharat models that runs on a standard CPU laptop.
- To provide a Gradio front end that shows the Hindi, improved phrasing and English outputs for a Hinglish input.
- To create a test set of 40 representative sentences and evaluate the pipeline with CER, word accuracy, chrF and BLEU.
- To analyse the failures, apply targeted fixes and measure their effect.

1.5 METHODOLOGY

The work was carried out in four steps. In the first step, the baseline pipeline was built: Roman Hinglish is transliterated with IndicXlit and the Hindi output is translated into English with the distilled 200M IndicTrans2 model. Both models are used as released and no model was trained.

In the second step, a test set of 40 Hinglish sentences was written in seven categories (short, long, English-mixed, names, numbers, slang and spelling variants), each with a reference Devanagari sentence and a reference English translation. The references were drafted with the help of an AI assistant without running the models and were then reviewed manually.

In the third step, the baseline was evaluated using character error rate and word accuracy for the Hindi output, and chrF and BLEU for the English output, both with strict scoring and with normalised scoring that ignores nukta and chandrabindu variants. The failures were grouped into categories.

In the fourth step, two rule-based fixes were added based on the error analysis: normalisation of informal Roman spellings before transliteration, and conversion of Devanagari numerals back to ASCII digits afterwards. The evaluation was repeated and the results were compared with the baseline. An optional phrasing step using a hosted language model, behind a swappable backend, completes the pipeline.

1.6 INNOVATION

- A modular pipeline in which each stage can be replaced independently; the phrasing backend is selected by a setting, so Param1 or another model can be substituted without changing other code.
- A lightweight normalisation layer that fixes informal spellings and numerals before and after the neural models, with a switch that reproduces the baseline.
- A reproducible CPU-only setup on Windows with WSL, including documented workarounds for fairseq and related dependencies.
- An evaluation that reports both strict and normalised scores, so that valid spelling variants are separated from genuine errors.

1.7 SUMMARY

This chapter introduced the problem of processing Roman-script Hinglish, the motivation and objectives, and the four-step methodology. The next chapter reviews the models and resources on which the project is based.

CHAPTER 2: LITERATURE REVIEW

2.1 LITERATURE REVIEW

This chapter describes the models and datasets used or considered in the project. Only general descriptions are given, and formal citations are [citations to be added].

IndicXlit is a transliteration model released by AI4Bharat that converts Roman-script text into the native scripts of several Indian languages, including Hindi. It is distributed through the ai4bharat-transliteration package and was used for the first stage of the pipeline [1].

IndicTrans2 is a family of translation models for Indian languages released by AI4Bharat. The distilled checkpoint ai4bharat/indictrans2-indic-en-dist-200M has about 0.2 billion parameters, an MIT licence, and translates from Indian languages into English. It is a gated repository on Hugging Face, so the terms of access must be accepted before download. It was used for the final stage of the pipeline [2].

Aksharantar is an AI4Bharat dataset of word pairs in Roman and native scripts intended for transliteration research. The project brief lists it as a source for transliteration evaluation, but it was not used in this work [3].

BharatGen Param1 is a language model from the BharatGen initiative and is the model named in the project brief for improving phrasing. It was not run in this project because of its memory requirements relative to the available hardware [4]. A hosted language model was used instead, behind a swappable backend.

Table 2.1: Summary of resources reviewed

Resource	Role in project	Strengths and weaknesses (observed in this project)	Tool used	Limitations
IndicXlit [1]	Roman to Devanagari transliteration	+ Runs on CPU; good on short and English-mixed sentences– Weak on SMS spellings, names and loanwords	fairseq-based transformer	Hard to install on Windows
IndicTrans2 dist-200M [2]	Hindi to English translation	+ Small enough for CPU (about 1.4 GB peak RAM)– Passes transliteration errors into English	transformers, IndicTransToolkit	Gated access; pinned library versions
Aksharantar [3]	Transliteration evaluation data	Listed in the brief	Not used	Not used in this work
BharatGen Param1 [4]	Phrasing improvement	Named in the brief	Not run	Too large for the available hardware
Hosted LLM (Gemini REST)	Phrasing improvement	+ Easy to call– Server errors (503) and timeouts occurred	requests	External dependency; not evaluated

The strengths and weaknesses in Table 2.1 describe observations made in this project, and are not general claims about the models.

CHAPTER 3: REQUIREMENTS

In this chapter, the system requirements for the Hinglish Smart Writing Assistant are outlined. Both functional and non-functional requirements, as well as hardware and software requirements necessary for the implementation of the system, are presented.

3.1 FUNCTIONAL REQUIREMENTS

The functional requirements describe the key operations of the system.

- Accept Roman-script Hinglish text entered by the user.
- Normalise common informal spellings before transliteration.
- Convert the text into Devanagari Hindi using IndicXlit and convert Devanagari numerals to ASCII digits.
- Optionally improve the phrasing of the Hindi text.
- Optionally translate the Hindi text into English using IndicTrans2.
- Show the Hindi, improved phrasing and English outputs in a web interface with example sentences.
- Handle empty input and errors without crashing, and fall back to the unchanged text if the phrasing step fails.
- Provide an evaluation script that scores the pipeline on the test set.

3.2 NON-FUNCTIONAL REQUIREMENTS

- Runs on a CPU-only laptop; the measured peak memory during translation was about 1.4 GB.
- Low delay: three sentences were translated in under four seconds in a test run.
- Models are loaded once and reused, so later requests are fast.
- Robustness: external API failures must not stop the application.
- Security: API keys are read from environment variables and are never stored in the repository.
- Reproducibility: pinned dependency versions and written setup notes.
- Modularity: each stage can be changed without affecting the others.

3.3 HARDWARE REQUIREMENTS

- Processor: Intel Core i5 or equivalent or better.
- RAM: minimum 8 GB recommended (the development machine has 15.8 GB).
- Storage: about 10 GB of free space for the environment and model files.
- GPU: not required; all models run on CPU.

3.4 SOFTWARE REQUIREMENTS

- Operating environment: Windows with WSL, Ubuntu 22.04.
- Programming language: Python 3.10.
- Deep learning framework: PyTorch (CPU build) with Hugging Face Transformers.
- Model libraries: ai4bharat-transliteration (with fairseq built from source) and IndicTransToolkit.
- Evaluation libraries: jiwer and sacrebleu.
- User interface framework: Gradio 4.44.1.

Table 3.1: Hardware and software configuration used

Item	Value
Processor	Intel Core i7-10510U, 4 cores, 8 threads
Memory	15.8 GB RAM
Graphics	Intel UHD and NVIDIA GeForce MX-series (not used)
Operating environment	Windows with WSL, Ubuntu 22.04
Python	3.10
PyTorch	2.14.1 (CPU build)
transformers / tokenizers	4.40.2 / 0.19.1
Gradio	4.44.1
Evaluation libraries	jiwer 4.0.0, sacrebleu 2.6.0

CHAPTER 4: MODULE DESIGN & ARCHITECTURE

This chapter presents the design and architecture of the Hinglish Smart Writing Assistant. It describes the overall structure, followed by a detailed design section that covers the individual modules and their interactions.

4.1 HIGH-LEVEL DESIGN

The design is a linear pipeline in which each stage builds on the previous one. Modularity was a priority, so that every stage can be changed or replaced without affecting the others.

4.1.1 SYSTEM ARCHITECTURE

The Roman-script input is first normalised and then transliterated into Devanagari by IndicXlit. Devanagari numerals in the output are converted to ASCII digits. An optional phrasing step then rewrites the Hindi text, and IndicTrans2 translates the result into English. A Gradio interface calls this pipeline and displays the three outputs.

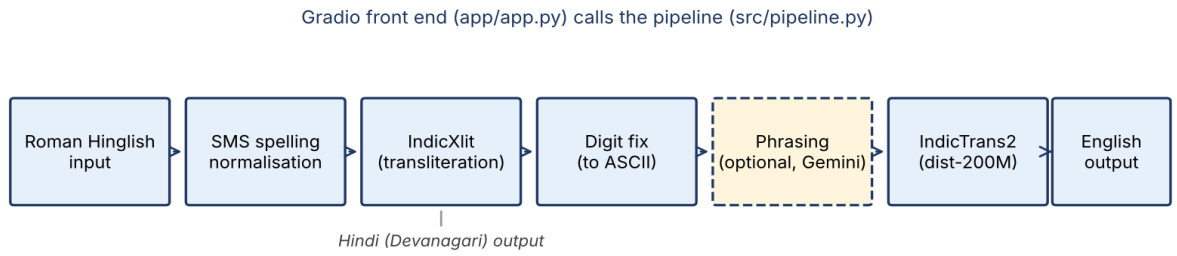


Fig 4.1: System Architecture

4.2 DETAILED DESIGN

The detailed design focuses on running on modest hardware, tolerating failures of external services, and keeping the baseline reproducible.

4.2.1 FUNCTIONAL DESCRIPTION OF THE MODULES

1. Normalisation Module (src/normalize.py)

A small dictionary maps informal Roman spellings to standard forms before transliteration, applied to whole lowercase words only (for example nhi to nahi, pta to pata, mai to main, thik to theek, hu to hoon). Names and capitalised words are left untouched. A second function converts Devanagari numerals to ASCII digits after transliteration, so that “500” is not turned into “५००”. The module can be switched off with the HINGLISH_NORMALIZE setting to reproduce the baseline.

2. Transliteration and Translation Module (src/pipeline.py)

The transliterate() function applies normalisation, calls the IndicXlit engine for Hindi and fixes the digits. The translate_to_english() function uses the distilled IndicTrans2 model with its tokenizer and the IndicTransToolkit processor. Each model is loaded once on first use and cached. The run_pipeline() function chains the stages.

3. Phrasing Module

The improve_phrasing() function sends the Hindi sentence to a hosted language model through its REST interface, with the instruction: rewrite the Hindi sentence in clear, natural Hindi, keep meaning, names and numbers exactly, add nothing, and return only the sentence. The PHRASING_BACKEND setting selects the backend (gemini or none). The key is read from an environment variable and never stored in the repository. A request timeout is applied, and on any error, missing key or unusable response the function returns the input text unchanged.

4. Front-End Module (app/app.py)

The Gradio application provides an input box, checkboxes for “Improve phrasing” and “Translate to English”, a Convert button, five clickable example sentences and three output boxes. Both models are preloaded at start-up, and pipeline errors are caught and shown as a friendly message. Screenshots are given in Appendix 1.

5. Evaluation Module (eval/run_eval.py)

The evaluation script runs all test cases through the pipeline and computes CER, word accuracy and exact match for the Hindi output, and chrF and BLEU for the English output, overall and by category. Scores are computed twice for Hindi: strictly, and after normalising nukta, chandrabindu, punctuation and spaces in both texts. Per-row results are written to a CSV file, and the ten worst rows by CER are listed.

4.3 DESIGN DECISIONS

- CPU-only execution: the distilled IndicTrans2 checkpoint and IndicXlit run without a GPU.
- Windows Subsystem for Linux: the fairseq library on which IndicXlit depends does not build reliably on Windows, so the Python environment runs inside Ubuntu 22.04 under WSL.
- Why Param1 was not run locally: it is a multi-billion-parameter model, and the development laptop (about 16 GB RAM, GeForce MX-series graphics with limited

video memory) cannot run it at usable speed. Running Param1 on a cloud GPU remains future work.

- Why the Google GenAI SDK was not used: a dry-run installation showed it would upgrade pydantic and websockets to versions incompatible with the pinned Gradio 4.44.1 stack, so the phrasing call uses the requests library that was already installed.

CHAPTER 5: IMPLEMENTATION

This chapter describes how the Hinglish Smart Writing Assistant was implemented, including the data used, the language and platform choices, the libraries and the problems that were solved during setup.

5.1 DATASET DESCRIPTION

No model was trained in this project, so no training dataset was used. A test set of 40 Hinglish sentences was created for evaluation, with a reference Devanagari sentence and a reference English translation for each. The references were drafted with the help of an AI assistant, without running the models, and were then reviewed manually by [to be filled]. The drafting followed fixed rules: chandrabindu before verb endings (for example जाएँगे), no nukta, English loanwords written phonetically in Devanagari, and numbers kept as digits. The complete test set is given in Appendix 2.

Table 5.1: Test set categories

Category	Row ids	Number of cases
short	1–6	6
long	7–12	6
english_mix	13–19	7
names	20–24	5
numbers	25–29	5
slang	30–35	6
spelling	36–40	5

The Aksharantar dataset named in the project brief was not used; adding an Aksharantar-based transliteration test is listed as future work.

5.2 PROGRAMMING LANGUAGE SELECTION

Python 3.10 was used for the whole project. It is the standard language for deep learning and has first-class support in PyTorch and Hugging Face Transformers. Version 3.10 was chosen because the fairseq-based IndicXlit stack and the pinned library versions were tested with it.

5.3 PLATFORM SELECTION

The application was developed and run on a Windows laptop using Windows Subsystem for Linux with Ubuntu 22.04, because fairseq could not be built reliably on Windows. Code was kept in a GitHub repository, models were downloaded from the Hugging Face Hub, and an

AI coding assistant (Google Antigravity) was used to help write code, which the team reviewed and tested.

Table 5.2: Libraries used in the project

Library	Purpose
torch (CPU build)	Neural network inference
transformers, sentencepiece, tokenizers	Loading and running the IndicTrans2 model
IndicTransToolkit	Pre- and post-processing for IndicTrans2
ai4bharat-transliteration, fairseq	IndicXlit transliteration engine
indic-nlp-library	Indic text processing used by IndicXlit
requests	REST call for the optional phrasing step
jiwer	Character and word error rates
sacrebleu	chrF and BLEU scores
numpy, pandas	Numerical computation and data handling
gradio	Web user interface

Table 5.3: Tools used in the project

Tool	Purpose
WSL (Ubuntu 22.04)	Linux environment for building and running the models
GitHub	Version control and repository hosting
Hugging Face Hub	Model download (gated IndicTrans2 checkpoint)
Google AI Studio	API key and model selection for the phrasing step
Google Antigravity	AI-assisted code generation, reviewed by the team

5.4 ENVIRONMENT AND DEPENDENCY CHALLENGES

Table 5.4: Dependency problems and solutions

Problem	Solution
fairseq failed to build from PyPI (missing version file).	Cloned the official fairseq repository and installed it from source with --no-build-isolation inside WSL, with pinned pip, setuptools and numpy.
ai4bharat-transliteration pulled in server and TensorFlow dependencies.	Installed with --no-deps and only the required runtime packages; a small patch to the urduhack package avoided importing TensorFlow.
IndicTransToolkit failed with newer transformers releases.	Pinned transformers 4.40.2 and huggingface-hub 0.23.5.
IndicTrans2 repository is gated (HTTP 401).	Accepted the terms on the Hugging Face model page and logged in from the command line.
Gradio 4.44.1 crashed with “unhashable type: dict” with newer fastapi and starlette.	Pinned fastapi 0.112.2, starlette 0.38.6 and pydantic 2.10.6.

5.5 SUMMARY

This chapter described the test data, the language and platform selection, the libraries and tools, and the dependency problems that were solved. The full setup procedure is recorded in the repository (docs/SETUP_NOTES.md) and is outlined in Appendix 3.

CHAPTER 6: RESULTS

6.1 EXPERIMENTAL RESULTS AND TESTING

The pipeline was evaluated on the 40 test cases described in Chapter 5. Two runs were made, both with the phrasing step off: a baseline run with normalisation off, and a second run with normalisation on.

6.1.1 Baseline Results

In the baseline run the strict CER was 7.00%, the strict word accuracy 81.85%, the corpus chrF 65.18 and the corpus BLEU 38.53. Short sentences and sentences with English words were handled best (strict CER of 1.16% and 2.55%). Informal spellings were the weakest category (20.31%), followed by numbers (14.75%).

Table 6.1: Baseline results by category (normalisation off)

Category	n	CER strict	CER norm.	WAcc strict	WAcc norm.	Exact match	chrF	BLEU
english_mix	7	2.55%	2.55%	95.56%	95.56%	71.4%	59.73	30.82
long	6	5.75%	4.79%	77.61%	82.09%	0.0%	73.09	50.20
names	5	5.42%	5.45%	86.11%	86.11%	0.0%	81.50	45.67
numbers	5	14.75%	13.93%	66.67%	70.37%	0.0%	63.45	37.28
short	6	1.16%	1.16%	95.83%	95.83%	83.3%	63.52	46.04
slang	6	8.63%	8.96%	81.82%	81.82%	50.0%	40.46	20.97
spelling	5	20.31%	20.31%	56.25%	56.25%	20.0%	62.11	23.40

6.1.2 Results with Normalisation

Table 6.2: Results by category with normalisation

Category	n	CER strict	CER norm.	WAcc strict	WAcc norm.	Exact match	chrF	BLEU
english_mix	7	2.55%	2.55%	95.56%	95.56%	71.4%	59.73	30.82
long	6	5.75%	4.79%	77.61%	82.09%	0.0%	73.09	50.20
names	5	5.42%	5.45%	86.11%	86.11%	0.0%	81.50	45.67
numbers	5	5.74%	4.92%	85.19%	88.89%	20.0%	63.45	37.28
short	6	1.16%	1.16%	95.83%	95.83%	83.3%	63.52	46.04
slang	6	7.91%	8.21%	84.85%	84.85%	50.0%	40.46	20.97
spelling	5	3.12%	3.12%	87.50%	87.50%	60.0%	71.93	49.44

6.1.3 Before and After

Table 6.3: Overall results, baseline versus normalised

Metric	Baseline	Normalised	Change
Strict CER	7.00%	4.88%	−2.12 points
Normalised CER	6.67%	4.54%	−2.13 points

Metric	Baseline	Normalised	Change
Strict word accuracy	81.85%	86.29%	+4.44 points
Normalised word accuracy	83.47%	87.90%	+4.43 points
Strict exact match	14 / 40	17 / 40	+3
Corpus chrF	65.18	65.58	+0.40
Corpus BLEU	38.53	39.50	+0.97

Table 6.4: Strict CER by category, before and after

Category	Baseline	Normalised	Change
spelling	20.31%	3.12%	−17.19
numbers	14.75%	5.74%	−9.01
slang	8.63%	7.91%	−0.72
long	5.75%	5.75%	0.00
names	5.42%	5.42%	0.00
english_mix	2.55%	2.55%	0.00
short	1.16%	1.16%	0.00

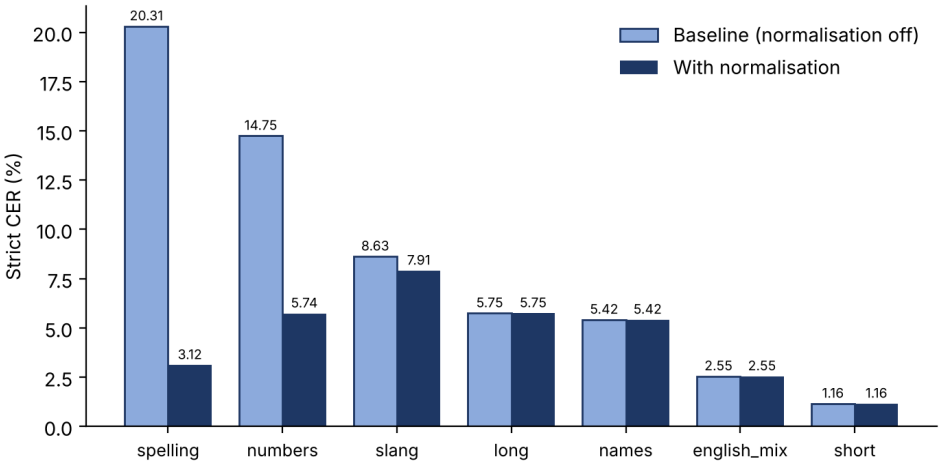


Fig 6.1: Strict CER by category, baseline versus normalised

The spelling category improved from the weakest to one of the strongest results, and its chrF rose from 62.11 to 71.93, which shows that cleaner Hindi also gave better English translations. The numbers category improved because Devanagari numerals no longer counted as errors against the digit references. Categories not affected by the two rules did not change.

6.1.4 Representative Examples

Table 6.5: Representative examples

Row	Input	Reference Hindi	Output Hindi	English output
25	mujhe 500 rupees chahiye	मुझे 500 रुपये चाहिए	baseline: मुझे ५०० रुपीस चाहिए; normalised: मुझे 500 रुपीस चाहिए	I need 500 rupees.
29	bill 1250 rupees ka aaya hai	बिल 1250 रुपये का आया है	baseline: बिल १२५० रुपीस का आया है; normalised: बिल 1250 रुपीस का आया है	The bill has come for 1250 rupees.
37	mujhe nhi pta	मुझे नहीं पता	baseline: मुझे नही पीटीए	I don't have a PTA. (baseline)
39	mai thik hu	मैं ठीक हूँ	baseline: मई थिक हू	May is fine. (baseline)
12	hamare college mein is saal naya library ban raha hai	हमारे कॉलेज में इस साल नई लाइब्रेरी बन रही है	हमारे कॉलेज में इस साल नया लाइब्रेरी बन रहा है	A new library is coming up in our college this year
14	meeting ka time change ho gaya hai	मीटिंग का टाइम चेंज हो गया है	मीटिंग का टाइम चंगे हो गया है	The time of the meeting has become good.
22	Amit ki shaadi Delhi mein hai	अमित की शादी दिल्ली में है	अमित की शादी डेल्ही में है	Amit's wedding is in Delhi.
23	Sneha aur Karthik kal Chennai ja rahe hain	स्नेहा और कार्तिक कल चेन्नई जा रहे हैं	स्नेह और कार्तिक कल चेन्नई जा रहे हैं	Sneh and Kartik are leaving for Chennai tomorrow.

The normalised outputs for rows 37 and 39 are [to be filled from eval/results_normalized.csv].

6.1.5 Error Analysis

The failures observed in the 40 test cases fall into the categories in Table 6.6. The full analysis, with further examples, is in docs/failure_analysis.md in the repository.

Table 6.6: Failure categories

Category	Rows	Likely cause	Possible fix
Informal spellings	37, 39, 38	"pta" read as the abbreviation PTA, "mai" as the month.	Spelling normalisation (applied).
Error propagation	37, 38, 39	Wrong Hindi leads to wrong English, for example "I don't have a PTA".	Fix the first stage.
Devanagari numerals	25, 28, 29, 34	IndicXlit converts digits to ५००.	Digit conversion (applied).
Loanword spelling	25, 32, 34	Loanwords spelled by sound (रुपीस, आईडीईए, मिन्यूट).	Small loanword dictionary.
Names and places	21, 22, 23, 24	Final vowels dropped (प्रिय, स्नेह); Delhi written as डेल्ही; Dr as डीआर.	Name and city dictionary.
Ambiguous words	14	"change" read as चंगे, giving "become good".	Context-aware model or dictionary.
Gender agreement	12	Roman text does not mark gender (नया instead of नई).	Language model with grammar context.
Missing chandrabindu	36	"nahi" gives नही instead of नहीं.	Post-edit rule or normalised scoring.
Translation style	34, 38	Valid English that differs from the single reference.	Multiple references.

Row 10 produced बेहें in place of बहन, which the translation model rendered as "sister-in-law", a further example of error propagation. Normalisation fixed only the first three categories.

6.1.6 Testing of the Phrasing Step

The phrasing step was implemented and tested with unit tests that use mocked responses for the missing-key, failed-request and timeout cases. In live tests, the hosted model repeatedly returned server-unavailable (503) errors and read timeouts, and the application returned the unchanged text as designed. The phrasing step was therefore not included in the quantitative evaluation.

6.2 EVALUATION METRICS

- Character error rate (CER): character-level edit distance between output and reference, divided by the reference length (lower is better).
- Word accuracy: one minus the word error rate.
- Exact match: the number of sentences whose output equals the reference.
- chrF and BLEU (sacreBLEU): similarity of the English translation to the reference translation.
- Strict scoring compares the text as it is. Normalised scoring first removes the nukta, maps chandrabindu to anusvara and removes punctuation and extra spaces in both texts, so that valid spelling variants are not counted as errors.

6.3 SUMMARY

The main results are as follows:

- Baseline strict CER of 7.00% and word accuracy of 81.85% on 40 test cases.
- After normalisation, strict CER of 4.88% and word accuracy of 86.29%.
- Spelling CER fell from 20.31% to 3.12% and numbers CER from 14.75% to 5.74%.
- Names, English loanwords, ambiguous words and gender agreement remain the main sources of error.

CHAPTER 7: CONCLUSION

7.1 DISCUSSION

The pipeline performed best on short sentences and sentences with English words, and worst on informal spellings and numerals, which were improved substantially by simple rules. The comparison shows that careful pre- and post-processing around pretrained models can give large gains at no training cost. Errors in transliteration carried over into translation, so improving the first stage also improved the English output. The remaining errors, such as dropped final vowels in names, phonetic spellings of loanwords and the choice between नया and नई, need context or knowledge that the transliteration model does not have.

7.2 LIMITATIONS AND THREATS TO VALIDITY

- Small test set: 40 sentences give only a rough estimate, and some categories contain five cases.
- References were drafted with an AI assistant and reviewed manually; a single reference per sentence penalises valid spelling variants.
- Overfitting risk: the normalisation dictionary was designed after examining the same 40 cases on which it was evaluated. A held-out set is needed to confirm the gain. The rule mapping “h” to “hai” could also affect initials.
- Results come from a single run.
- The phrasing step was not evaluated because the hosted model returned errors during testing.
- The Aksharantar dataset named in the brief was not used, and Param1 was not run.

7.3 CONCLUSION

The project delivered a working end-to-end Hinglish writing assistant that runs on a CPU laptop using AI4Bharat IndicXlit and IndicTrans2, with a Gradio front end and a quantitative evaluation. The baseline reached a strict CER of 7.00% and a word accuracy of 81.85%. Targeted normalisation reduced the CER to 4.88% and raised word accuracy to 86.29%, mainly by fixing informal spellings and numerals. The application, test set, evaluation scripts and documentation are available in the project repository.

7.4 FUTURE ENHANCEMENT

- Run Param1 on a cloud GPU, or complete an evaluation of the hosted phrasing step.
- Add an Aksharantar-based transliteration test.
- Add dictionaries for common loanwords and city names.
- Evaluate on a held-out test set with several references per sentence, and add human evaluation.
- Extend the system to other Indian languages and to speech input.

REFERENCES

[1] AI4Bharat, IndicXlit and Indic transliteration. https://github.com/AI4Bharat/IndianNLP-Transliteration

[2] AI4Bharat, IndicTrans2. https://github.com/AI4Bharat/IndicTrans2 ; model: https://huggingface.co/ai4bharat/indictrans2-indic-en-dist-200M

[3] AI4Bharat, Aksharantar dataset. https://github.com/AI4Bharat/IndianNLP-Transliteration

[4] BharatGen organisation on Hugging Face. https://huggingface.co/bharatgenai

APPENDICES

APPENDIX 1: SCREENSHOTS

The following screenshots show the Gradio front end of the Hinglish Smart Writing Assistant running locally at http://localhost:7860.

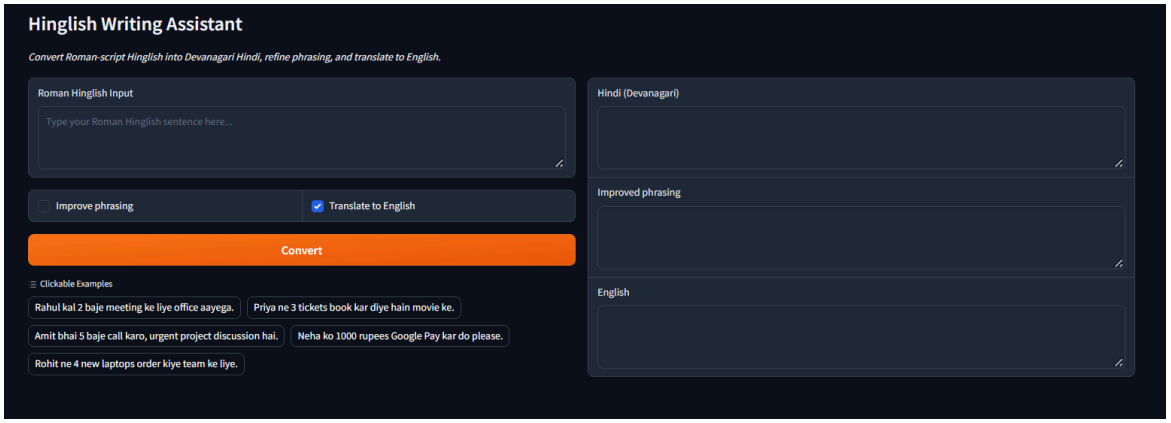


Fig A1.1: Start screen of the application with the input box, option checkboxes, Convert button, example sentences and three output boxes

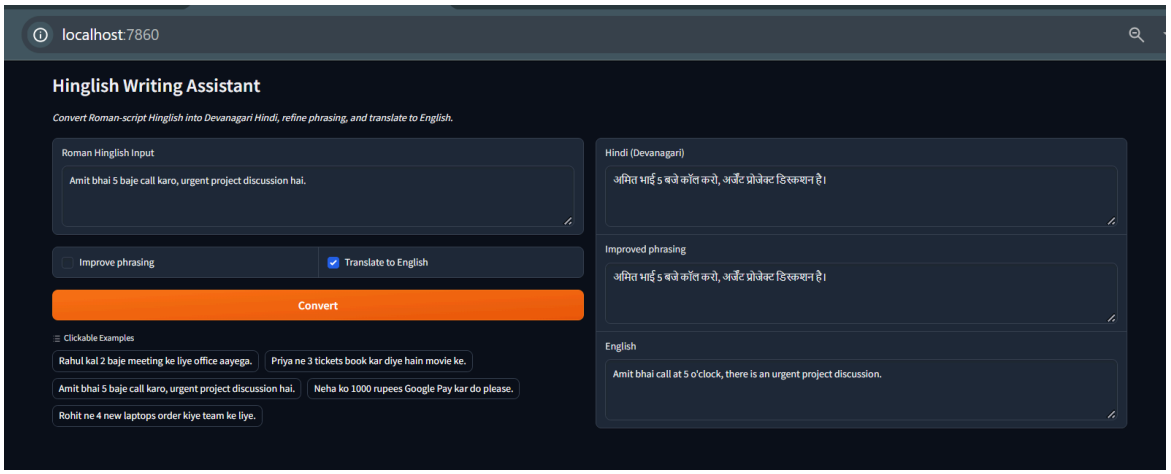


Fig A1.2: Example with "Improve phrasing" off and "Translate to English" on: Hindi output and English translation

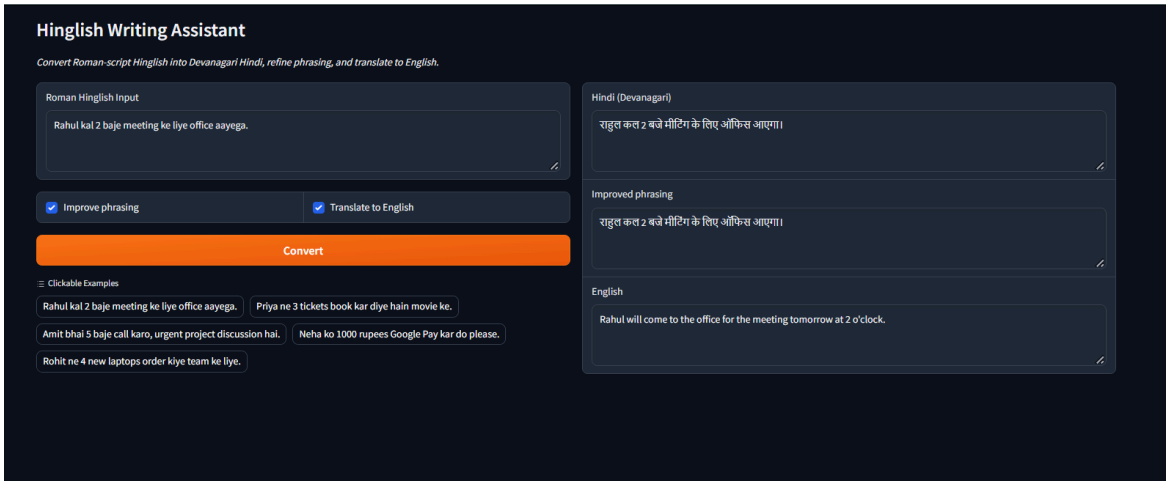


Fig A1.3: Example with both options on; the phrasing step returned the Hindi text unchanged because the hosted model was unavailable (fallback behaviour)

APPENDIX 2: TEST CASES

Table A2.1: The 40 test cases

Id	Category	Hinglish input	Reference Hindi	Reference English
1	short	main ghar ja raha hoon	मैं घर जा रहा हूँ	I am going home
2	short	aap kaise ho	आप कैसे हो	How are you?
3	short	kal milte hain	कल मिलते हैं	We will meet tomorrow
4	short	mujhe bhookh lagi hai	मुझे भूख लगी है	I am hungry
5	short	yeh kitne ka hai	यह कितने का है	How much is this?
6	short	mujhe neend aa rahi hai	मुझे नींद आ रही है	I am feeling sleepy
7	long	mujhe lagta hai ki hamein exam ki taiyari abhi se shuru kar deni chahiye	मुझे लगता है कि हमें एग्ज़ाम की तैयारी अभी से शुरू कर देनी चाहिए	I think we should start preparing for the exam right now
8	long	agar kal baarish hui toh hum picnic par nahi jaayenge	अगर कल बारिश हुई तो हम पिकनिक पर नहीं जाएँगे	If it rains tomorrow, we will not go for a picnic
9	long	professor ne bola ki assignment Friday tak submit karna hai	प्रोफेसर ने बोला कि असाइनमेंट फ्राइडे तक सबमिट करना है	The professor said that the assignment has to be submitted by Friday
10	long	meri behen ne kaha ki woh shaam ko market jaayegi aur sabzi layegi	मेरी बहन ने कहा कि वह शाम को मार्केट जाएगी और सब्ज़ी लाएगी	My sister said that she will go to the market in the evening and bring vegetables
11	long	jab tak tum nahi aate main yahin tumhara intezaar karunga	जब तक तुम नहीं आते मैं यहीं तुम्हारा इंतजार करूँगा	I will wait for you right here until you come
12	long	hamare college mein is saal naya library ban raha hai	हमारे कॉलेज में इस साल नई लाइब्रेरी बन रही है	A new library is being built in our college this year
13	english_mix	mera laptop kharab ho gaya hai	मेरा लैपटॉप खराब हो गया है	My laptop has stopped working
14	english_mix	meeting ka time change ho gaya hai	मीटिंग का टाइम चेंज हो गया है	The meeting time has changed
15	english_mix	please mujhe project ki file email kar do	प्लीज मुझे प्रोजेक्ट की फाइल ईमेल कर दो	Please email me the project file
16	english_mix	bus stand pe milte hain	बस स्टैंड पे मिलते हैं	Let's meet at the bus stand
17	english_mix	internet bahut slow chal raha hai	इंटरनेट बहुत स्लो चल रहा है	The internet is running very slow
18	english_mix	mobile ki battery low ho gayi hai	मोबाइल की बैटरी लो हो गई है	The phone battery is low
19	english_mix	canteen mein lunch kar lete hain	कैंटीन में लंच कर लेते हैं	Let's have lunch in the canteen
20	names	Rahul Mysuru se aa raha hai	राहुल मैसूरु से आ रहा है	Rahul is coming from Mysuru
21	names	Priya ne Bangalore mein job li hai	प्रिया ने बैंगलोर में जॉब ली है	Priya has taken a job in Bangalore
22	names	Amit ki shaadi Delhi mein hai	अमित की शादी दिल्ली में है	Amit's wedding is in Delhi

Id	Category	Hinglish input	Reference Hindi	Reference English
23	names	Sneha aur Karthik kal Chennai ja rahe hain	स्नेहा और कार्तिक कल चेन्नई जा रहे हैं	Sneha and Karthik are going to Chennai tomorrow
24	names	Dr Sharma ka lecture kal Mysore University mein hai	डॉ. शर्मा का लेक्चर कल मैसूर यूनिवर्सिटी में है	Dr Sharma's lecture is at Mysore University tomorrow
25	numbers	mujhe 500 rupees chahiye	मुझे 500 रुपये चाहिए	I need 500 rupees
26	numbers	class 10 baje shuru hogi	क्लास 10 बजे शुरू होगी	The class will start at 10 o'clock
27	numbers	main 3 din baad wapas aaunga	मैं 3 दिन बाद वापस आऊँगा	I will return after 3 days
28	numbers	is semester mein 5 subjects hain	इस सेमेस्टर में 5 सब्जेक्ट्स हैं	There are 5 subjects in this semester
29	numbers	bill 1250 rupees ka aaya hai	बिल 1250 रुपये का आया है	The bill has come to 1250 rupees
30	slang	kya haal hai bhai	क्या हाल है भाई	What's up, bro?
31	slang	chal yaar, baad mein baat karte hain	चल यार, बाद में बात करते हैं	Come on friend, let's talk later
32	slang	arre wah, mast idea hai	अरे वाह, मस्त आइडिया है	Oh wow, great idea
33	slang	kuch nahi yaar, bas timepass	कुछ नहीं यार, बस टाइमपास	Nothing friend, just passing time
34	slang	abhi aata hoon bhai, bas 2 minute	अभी आता हूँ भाई, बस 2 मिनट	Coming right away brother, just 2 minutes
35	slang	bindaas bol, kya scene hai	बिंदास बोल, क्या सीन है	Go ahead, say it, what's the deal?
36	spelling	mujhe nahi pata	मुझे नहीं पता	I don't know
37	spelling	mujhe nhi pta	मुझे नहीं पता	I don't know
38	spelling	main theek hun	मैं ठीक हूँ	I am fine
39	spelling	mai thik hu	मैं ठीक हूँ	I am fine
40	spelling	kyaa tum aaj aaoge	क्या तुम आज आओगे	Will you come today?

APPENDIX 3: SETUP OUTLINE

The exact sequence is documented in docs/SETUP_NOTES.md and the README. In outline:

- Install WSL with Ubuntu 22.04 and create a Python 3.10 virtual environment at ~/venvs/hinglish.
- Install the CPU build of PyTorch from the official PyTorch CPU wheel index.
- Pin pip below 24.1, setuptools below 70 and numpy below 2; install Cython and wheel.
- Clone the official fairseq repository and install it with pip and --no-build-isolation.
- Install ai4bharat-transliteration with --no-deps, then its required runtime packages; apply the urduhack patch.
- Install the remaining packages with the pinned versions in requirements.txt.
- Create a Hugging Face account, accept the terms for ai4bharat/indictrans2-indic-en-dist-200M and log in with hf auth login.
- Run the application with python app/app.py and open http://localhost:7860.

APPENDIX 4: REPOSITORY STRUCTURE

```
hinglish-writing-assistant/
├── app/
│   └── app.py                      # Gradio web UI application
├── data/
│   └── test_cases.csv              # 40 curated test cases across 7 categories
├── docs/
│   ├── failure_analysis.md         # Comprehensive failure analysis & empirical report
│   └── SETUP_NOTES.md              # Detailed WSL setup guide & dependency workarounds
├── eval/
│   ├── results.csv                 # Baseline evaluation results (normalisation OFF)
│   ├── results_normalized.csv      # Normalised pipeline evaluation results (normalisation ON)
│   └── run_eval.py                 # Evaluation benchmark script (jiwer + sacrebleu)
├── src/
│   ├── __init__.py
│   ├── normalize.py                # SMS shorthand expansion & Devanagari digit conversion
│   └── pipeline.py                 # Core pipeline (transliterate, improve phrasing, translate)
├── tests/
│   ├── test_normalize.py           # Unit tests for text normalization
│   ├── test_phrasing.py            # Unit tests for phrasing backend
│   ├── test_pipeline.py            # Unit tests for end-to-end pipeline
│   └── test_translation_wsl.py     # Integration test for IndicTrans2 in WSL
├── .gitignore                      # Git ignore rules
├── PROJECT_BRIEF.md                # Mini-project specification & original requirements
├── README.md                       # Project documentation (this file)
└── requirements.txt                # Pinned package requirements
```